



Search



Politecnico
di Torino

Implementazione del Metodo delle Correnti di Maglia

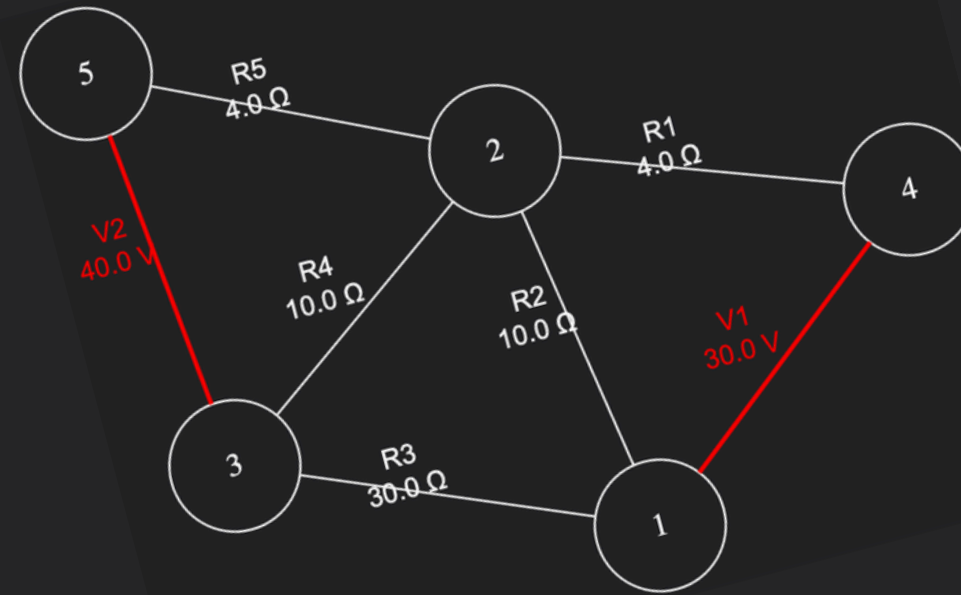
```
35
36  template <typename T>
37  √ std::vector<struttura_cicli<T> > cicli_fondamentali_dfs(const undirected_graph<T>& grafo, const T& nodo_sorgente)
38  {
39      lifo<T> stack;
40      undirected_graph<T> A = graph_visit(grafo, nodo_sorgente, stack); //albero dfs usando lo stack
41      undirected_graph<T> coalbero = grafo - A;    // coalbero C=G\T (archi in G ma non in T)
42      std::vector<struttura_cicli<T>> cicli;
43
44  √  for (const undirected_edge<T>& arco : coalbero.all_edges()) //per ogni arco (u,v) in C trova il percorso da u a
45      {
```

Corso di Programmazione e Calcolo Scientifico - 2025/2026

Alice Romeo, Martina Stivala, Eleonora Vacchieri

Obiettivo del progetto

Dato un input, in questo caso un circuito elettrico sotto forma di **netlist**, il codice deve trasformarlo in un **sistema lineare** e poi restituire il **valore delle tensioni e delle correnti su ciascun resistore**



Abbiamo suddiviso il progetto in **moduli indipendenti**: lettura della netlist, rappresentazione del circuito come grafo, ricerca delle maglie e costruzione del sistema lineare.

Idea generale:

Input circuito
↓
Costruzione grafo
↓
Ricerca dei cicli
↓
Costruzione delle matrici
↓
Risoluzione sistema lineare
↓
Output tensioni e correnti



Strutture dati e algoritmi di base

contenitori.hpp

Inizializzazione di due contenitori generici:

```
fifo<T> // queue  
lifo<T> // stack
```

Entrambi si avvalgono degli stessi metodi **put()**, **get()** e **empty()**, così la funzione di visita del grafo può essere la stessa (se viene passata una coda fa BFS, se viene passata una pila fa DFS)

graph.hpp

Scelte implementative:

undirected_graph<T>

per rappresentare il circuito come un grafo

undirected_edge<T>

per rappresentare gli archi

graph_visit.hpp

Contiene gli **algoritmi di visita** del grafo

prende:

- un grafo
- un nodo sorgente
- un contenitore

e durante la visita costruisce un albero di supporto (sottoinsieme di archi che collega tutti i nodi senza formare cicli). Serve per la costruzione del **coalbero**.



Strutture base

Per la memorizzazione dei circuiti abbiamo utilizzato:

```
struct struttura {  
    std::string type; -----> memorizza il tipo di componente e il suo numero progressivo  
    double value; -----> valore tensione/resistenza  
    int sign; -----> assume valore +1 se nodo1<nodo2, altrimenti ha valore -1  
    int nodo1;  
    int nodo2;  
};  
  
struct circuito{  
    undirected_graph<int> G; -----> grafo con archi pesati  
    std::map<undirected_edge<int>, struttura> componenti; ---> associa la componente  
    all'arco in maniera  
    immediata  
};
```



Ricerca delle maglie

`cicli_fondamentali.hpp``DePina.hpp`

la struttura utilizzata per l'implementazione di entrambi i metodi:

```
struct struttura_cicli {  
    std::vector<T> nodes;  
    std::set<unidirected_edge<T>> edges;  
};
```

Difficoltà riscontrate

- **ordinamento delle maglie:** strutture ordinate map/set alteravano il verso degli archi, creando problemi in B
→ `inline void orienta_maglie(std::vector<struttura_cicli<int>>& maglie);`
- **nodo partenza dei cicli fondamentali:** i cicli non partivano dal nodo con indice minimo
→ `std::rotate(ciclo.nodes.begin(), node_min , ciclo.nodes.end() - 1);`



Costruzione delle matrici

`metodi_correnti.hpp`

E' il file centrale del metodo, contiene la costruzione delle matrici.

MATRICE R

`Eigen::MatrixXd R = valori.asDiagonal();`
perchè le resistenze compaiono sulla diagonale

MATRICE B

Contiene 0, +1, -1 a seconda dell'appartenenza alla
maglia della resistenza e del verso in cui viene
attraversata dalla corrente

VETTORE v

vettore dei termini noti dei generatori



Difficoltà riscontrate

- **gestione dei segni:** probabilmente il problema maggiore del progetto

Risoluzione del sistema e calcolo finale

circuiti.cpp

scelte implementative:

per risolvere il sistema lineare $B'RBi = v$ usiamo l'implementazione del gradiente coniugato

```
risultati ris_dfs = gradiente_coniugato(A, v, x0_1, tolleranza, iter_max);  
Eigen::VectorXd i = ris_dfs.x;
```

```
Eigen::VectorXd tensioni = R * B * i;  
Eigen::VectorXd correnti = B * i;  
Eigen::VectorXd tensioni2 = R * B2 * i2;  
Eigen::VectorXd correnti2 = B2 * i2;
```



Difficoltà riscontrate

- trovare il metodo più adeguato per risolvere il sistema lineare

Netlist di prova e output finale

NETLIST:

```
V1 30.0 1 4
V2 40.0 3 5
R1 4.0 4 2
R2 10.0 1 2
R3 30.0 1 3
R4 10.0 3 2
R5 4.0 2 5
```

----->

-RISULTATI CICLI FONDAMENTALI (DFS):

R2: V = 22 volts, I = 2.2 amps
R3: V = -6 volts, I = -0.2 amps
R4: V = -28 volts, I = -2.8 amps
R1: V = 8 volts, I = 2 amps
R5: V = 12 volts, I = 3 amps

-RISULTATI DE PINA:

R2: V = 22 volts, I = 2.2 amps
R3: V = -6 volts, I = -0.2 amps
R4: V = -28 volts, I = -2.8 amps
R1: V = 8 volts, I = 2 amps
R5: V = 12 volts, I = 3 amps



Conclusione: come ci aspettavamo, i risultati con i due metodi sono uguali



Search



Politecnico
di Torino

Grazie per l'attenzione

```
17  risultati gradiente_coniugato(const Eigen::MatrixXd& A, const Eigen::VectorXd& b, const Eigen::VectorXd& x)
18
19      unsigned int n = b.size();
20      Eigen::VectorXd x = Eigen::VectorXd::Zero(n);
21      Eigen::VectorXd res = b - A * x;
22      Eigen::VectorXd p = res;
23      double res_norm_0 = res.norm();
24
25      unsigned int it=0;
26
27      //iterazione
```

Alice Romeo, Martina Stivala, Eleonora Vacchieri